

WhiteDavid23 Academy

Frontend System Design

Partner Portal / Admission CRM Platform

Frontend: Next.js + TypeScript

UI: Tailwind CSS

Architecture: Feature-oriented modular frontend

API: REST `/api/v1`

Primary Target: Desktop + Android/mobile responsive web

1. Frontend Responsibilities

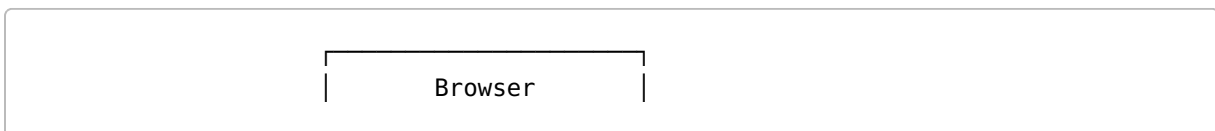
The frontend is responsible for:

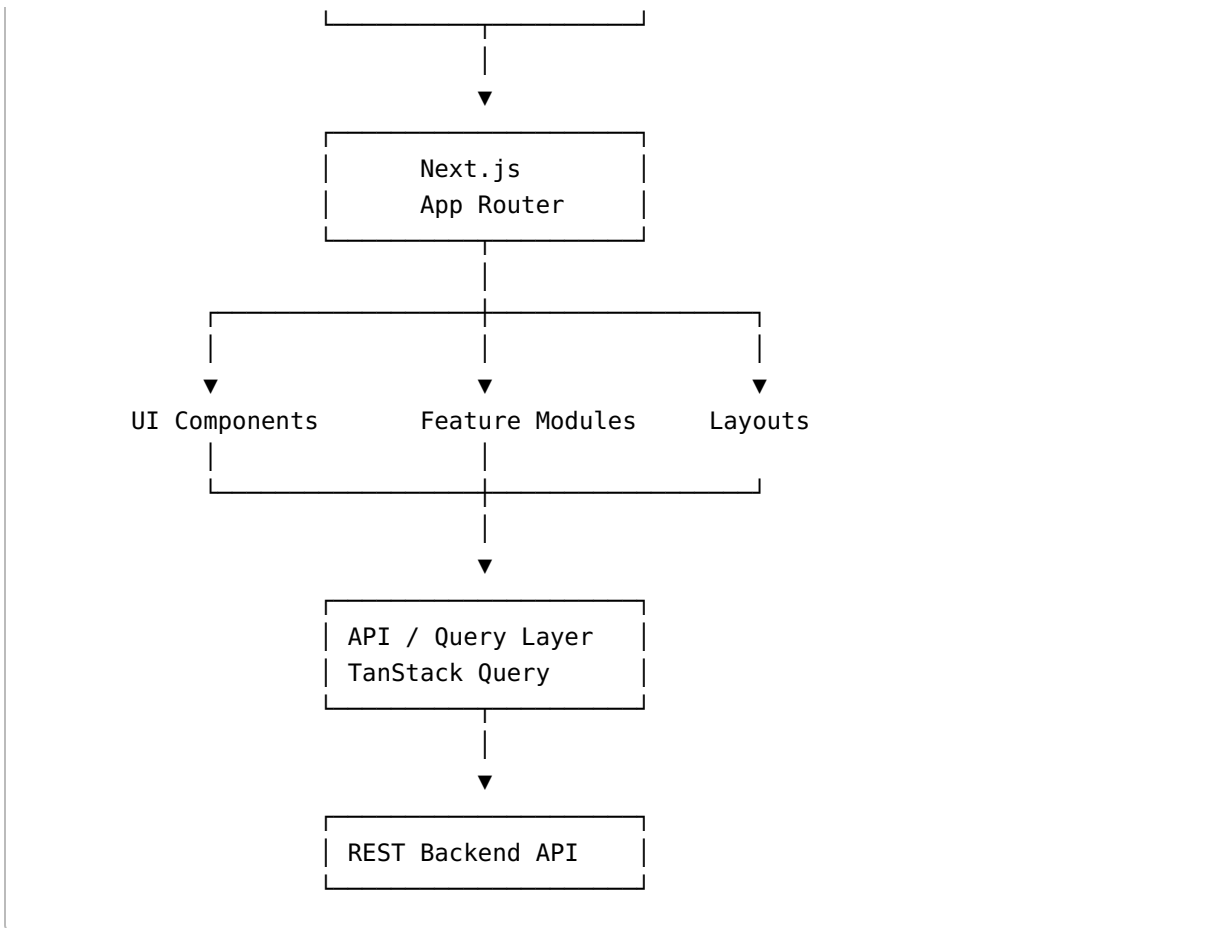
- user interface
- navigation
- responsive layouts
- forms
- client-side validation
- API communication
- loading states
- error states
- table/filter interfaces
- dashboard visualization
- permission-aware UI
- notifications
- user interactions
- accessibility
- responsive mobile experience

The frontend is **not responsible for enforcing security**.

Backend authorization remains authoritative.

2. Frontend Architecture





3. Recommended Frontend Stack

Next.js
TypeScript
Tailwind CSS

TanStack Query
React Hook Form
Zod

Recharts
Lucide Icons

Axios or fetch wrapper

Vitest/Jest

4. Frontend Directory Structure

```
frontend/
├── src/
│   ├── app/
│   │   ├── (auth)/
│   │   │   ├── login/
│   │   │   └── forgot-password/
│   │   ├── (dashboard)/
│   │   │   ├── dashboard/
│   │   │   ├── leads/
│   │   │   ├── follow-ups/
│   │   │   ├── admissions/
│   │   │   ├── courses/
│   │   │   ├── marketing/
│   │   │   ├── commissions/
│   │   │   ├── reports/
│   │   │   ├── notifications/
│   │   │   └── settings/
│   │   └── admin/
│   │       ├── partners/
│   │       ├── users/
│   │       └── audit-logs/
│   └── features/
│       ├── auth/
│       ├── dashboard/
│       ├── leads/
│       ├── followups/
│       ├── admissions/
│       ├── courses/
│       ├── marketing/
│       ├── commissions/
│       ├── reports/
│       ├── partners/
│       └── users/
```

```

|   |   | components/
|   |   | | ui/
|   |   | | layout/
|   |   | | tables/
|   |   | | forms/
|   |   | | charts/
|   |   | | feedback/
|   |   |
|   |   | lib/
|   |   | | api/
|   |   | | auth/
|   |   | | permissions/
|   |   | | utilities/
|   |   |
|   |   | hooks/
|   |   | types/
|   |   | schemas/
|   |   | constants/
|   |
|   | public/
|   | tests/
|   | package.json

```

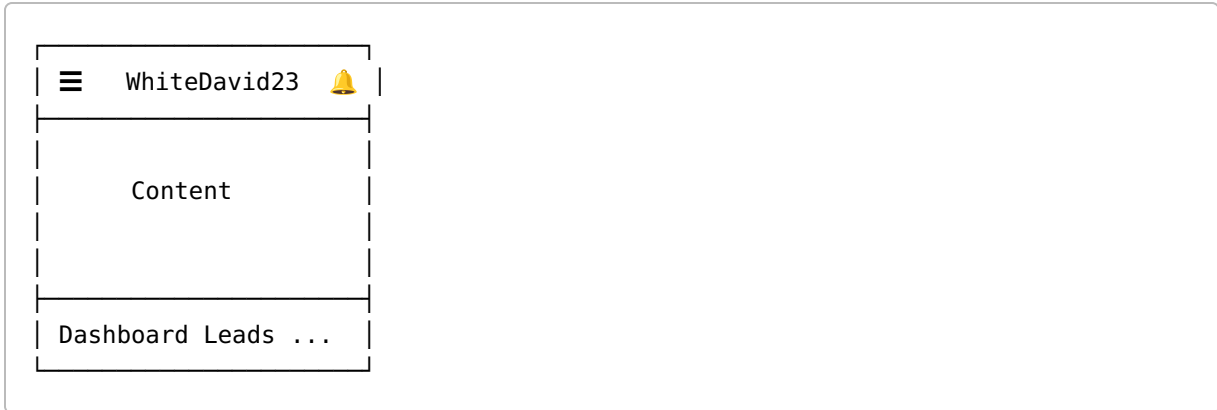
5. Application Layout

Authenticated users enter:

Topbar		Notifications
Dashboard Leads Follow-ups Admissions Courses Marketing Commission Reports Settings		Main Content



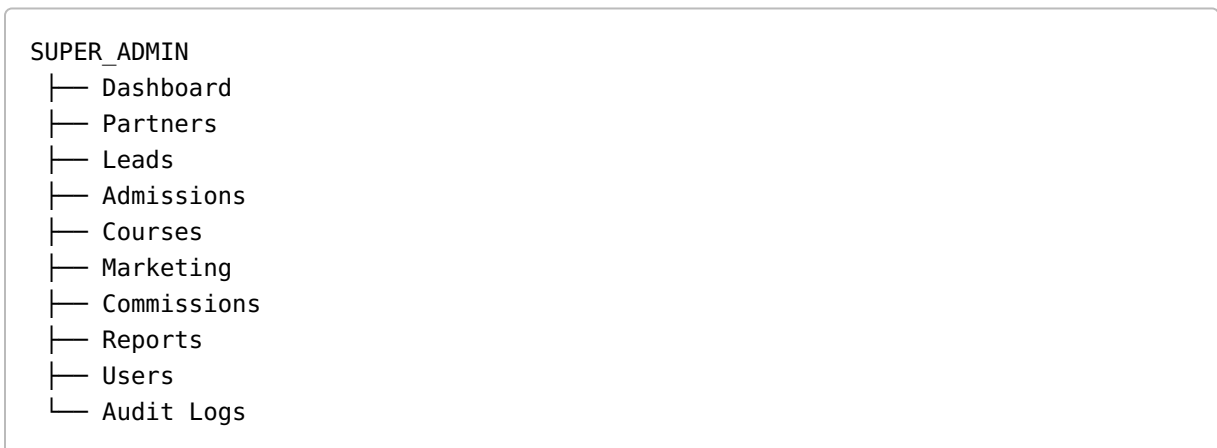
On mobile:



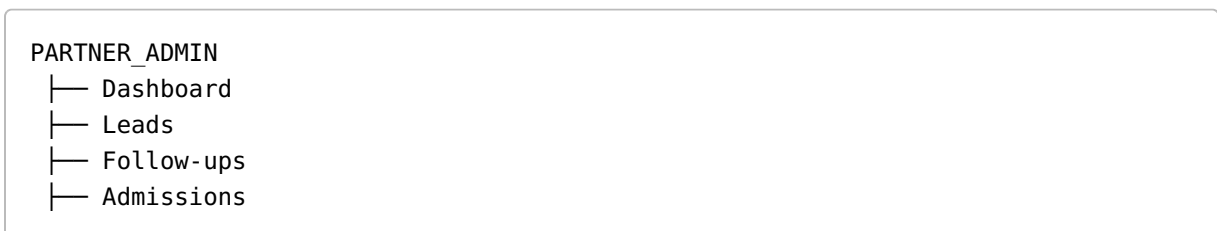
6. Role-Aware Navigation

Navigation is generated from permissions.

Example:



Partner:



```
├ Courses
├ Marketing
├ Commissions
├ Reports
└ Team
```

Counselor:

```
COUNSELOR
├ Dashboard
├ My Leads
├ Follow-ups
├ Admissions
├ Courses
└ Marketing
```

Hiding a navigation item is UX.

It is **not security**.

7. Authentication State

Frontend authentication should maintain:

```
user
role
permissions
partner
session
```

Conceptual:

```
AuthProvider
|
├ user
├ role
├ permissions
└ session state
```

On startup:

```
Browser
↓
Restore session
↓
GET /auth/me
↓
Load user
↓
Load permissions
↓
Render application
```

8. Route Protection

Protected routes:

```
/dashboard
/leads
/admissions
/reports
/settings
```

Unauthenticated user:

```
protected route
↓
not authenticated
↓
/login
```

Unauthorized user:

```
authenticated
↓
no required permission
↓
403 / Access Denied
```

9. Dashboard

Dashboard should be role-specific.

Super Admin

- Total Partners
- Total Leads
- Admissions
- Revenue
- Conversion
- Commission
- Partner Performance
- Course Performance

Partner Admin

- My Leads
- Hot Leads
- Today's Follow-ups
- Admissions
- Conversion
- Commission

Counselor

- Assigned Leads
- Today's Tasks
- Overdue Follow-ups
- Recent Activity

10. Lead List

Features:

- Search
- Filter
- Sort
- Pagination

Status filter
Priority filter
Course filter
Assigned user
Date range
Bulk actions

Example:

Search leads...		Filter	+ Add Lead	
Name	Phone	Course	Status	Owner
Rahul	...	Java	Hot	Ahmed
Sana	...	AI	Demo	Ayan

11. Lead Details

Lead detail page:

Rahul Kumar		HOT
Phone	WhatsApp	Email
Course: Java Backend		
Source: Website		
Budget: ₹50,000		
Next Follow-up: Tomorrow 10:00 AM		
Timeline		
Created		
↓		
Called		
↓		
Brochure Sent		
↓		

Demo Booked

12. Lead Form

Form fields:

Name *
Phone *
WhatsApp
Email
City
Course *
Source
Budget
Priority
Status
Follow-up date
Assigned user
Notes

Validation:

required fields
phone format
email format
valid dates
valid enum values

Use shared schemas where possible.

13. Follow-up Center

Main views:

Today's Follow-ups
Overdue
Upcoming

Completed
Snoozed

Calendar:

Monday
└─ 10:00 Rahul
└─ 11:30 Sana
└─ 15:00 Ahmed

Tuesday
└─ ...

Each task should provide quick actions:

Complete
Reschedule
Snooze
Call
WhatsApp
Add Note

14. Admissions UI

Admission list:

Student
Course
Partner
Fee
Payment
Status
Joining Date
Commission

Admission details:

Student Information
Course Information
Payment History

Documents
Verification
Commission
Activity

15. Course UI

Course cards:

Java Backend
6 Months
₹50,000
View Course
Brochure
Demo

Course details:

Overview
Duration
Fee
Benefits
Syllabus
Brochure
Demo
FAQ

16. Marketing Center

Categories:

Posters
Banners
Brochures
WhatsApp Templates
Captions

Videos
Testimonials

Actions:

Preview
Download
Share
Copy Text

17. Commission UI

Dashboard cards:

Earned
Pending
Approved
Paid

Commission table:

Admission
Course
Fee
Rate
Commission
Status
Date

Partner users only see their own commission data.

18. Reports UI

Reports should provide:

Date range
Partner
Course

```
Status
Source
City
```

Charts:

```
Lead funnel
Admission trend
Revenue trend
Partner performance
Course performance
Conversion rate
```

Exports:

```
CSV
Excel
PDF
```

Export actions should respect backend permissions.

19. API Integration Layer

Do not scatter raw API calls throughout components.

Bad:

```
page.tsx
fetch(...)
fetch(...)
fetch(...)
```

Preferred:

```
features/leads/api.ts

getLeads()
getLead()
createLead()
updateLead()
```

```
assignLead()  
changeLeadStatus()
```

Then:

```
React Component  
  ↓  
Feature API / Hook  
  ↓  
API Client  
  ↓  
Backend
```

20. TanStack Query Architecture

Use query caching for server state.

Example conceptual hooks:

```
useLeads()  
useLead(id)  
useCreateLead()  
useUpdateLead()  
useAssignLead()  
useFollowUps()  
useAdmissions()  
useDashboard()
```

Mutation:

```
Create Lead  
  ↓  
POST /leads  
  ↓  
Invalidate leads query  
  ↓  
Refresh list
```

21. State Management

Separate state into:

Server State

```
leads  
admissions  
courses  
reports  
notifications
```

Use TanStack Query.

Local UI State

```
modal open  
sidebar collapsed  
selected rows  
filters
```

Use React state.

Form State

Use React Hook Form.

Authentication State

Use a dedicated auth/session mechanism.

Avoid putting the entire application state into one global store.

22. Loading States

Every API-driven page should have:

```
Loading  
Success
```

Empty
Error

Example:

```
Loading:
Skeleton

Success:
Table

Empty:
"No leads yet"
"+ Add Lead"

Error:
"Unable to load leads"
"Retry"
```

23. Error Handling

API error:

```
{
  "status": 403,
  "message": "You do not have permission to perform this action"
}
```

Frontend:

```
403
↓
Permission error UI
```

Validation error:

```
fieldErrors
↓
Attach errors to form fields
```

24. Responsive Design

Breakpoints should support:

```
Mobile
Tablet
Desktop
Large desktop
```

Mobile priorities:

```
lead details
follow-ups
quick actions
admission updates
notifications
```

Large tables should become:

```
responsive cards
horizontal scroll
or
priority-column layouts
```

Do not simply shrink desktop tables until they become unusable.

25. Component Architecture

Shared UI:

```
Button
Input
Select
Modal
Dialog
Dropdown
Badge
Card
Table
```

Pagination
Tabs
Toast
Skeleton
EmptyState
ErrorState

Business components:

LeadCard
LeadStatusBadge
LeadTimeline
FollowUpCard
AdmissionStatusBadge
CommissionCard
PartnerCard

26. Design System

Define centrally:

Typography
Spacing
Radius
Shadows
Buttons
Forms
Status badges
Tables
Cards
Modals
Alerts

Status colors should have consistent semantic meaning:

success
warning
danger
info
neutral

Avoid arbitrary per-page styling.

27. Permission-Aware UI

Example:

```
if permission = LEAD_ASSIGN
  show "Assign Lead"

if permission = COMMISSION_VIEW
  show "Commission"

if permission = PARTNER_APPROVE
  show "Approve"
```

But backend still verifies:

```
POST /partners/:id/approve
```

The frontend should never assume that hiding a button grants security.

28. Optimistic Updates

Use carefully.

Good candidates:

```
mark notification read
toggle UI preference
```

Use caution with:

```
admission status
payment
commission
partner approval
lead transfer
```

Financial and sensitive state should generally wait for server confirmation.

29. Accessibility

Frontend should support:

```
keyboard navigation
visible focus
semantic HTML
labels
ARIA where necessary
sufficient contrast
screen-reader friendly forms
error announcements
```

30. Frontend Testing

Unit

Test:

```
components
formatters
validators
permission utilities
```

Integration

Test:

```
forms
API states
filters
tables
modals
```

E2E

Critical journeys:

```
Login
Create lead
Assign lead
Schedule follow-up
Convert admission
Record payment
View report
```

31. Frontend Performance

Use:

```
server rendering where appropriate
lazy loading
code splitting
image optimization
pagination
query caching
debounced search
virtualization for very large lists
```

Do not load thousands of leads into the browser just to filter them locally.

Filtering should normally happen through the API.

32. Frontend Security

Never put secrets in:

```
NEXT_PUBLIC_*
```

Anything public to the browser must be assumed visible.

Do not store sensitive authentication information carelessly in localStorage.

Prefer secure, appropriate cookie/token strategies.

33. Frontend Environment

Example:

```
NEXT_PUBLIC_API_URL=https://api.example.com
```

Only public configuration belongs in `NEXT_PUBLIC_*`.

Private secrets remain server-side.

34. Frontend Definition of Done

A feature is complete when:

- ✓ Page exists
- ✓ Responsive layout works
- ✓ API integration works
- ✓ Loading state exists
- ✓ Empty state exists
- ✓ Error state exists
- ✓ Form validation exists
- ✓ Permission-aware UI exists
- ✓ Accessibility considered
- ✓ Mobile tested
- ✓ E2E path tested

35. Frontend Architectural Principle

The frontend should follow:

```
Page
↓
Feature
↓
Hook
```

↓
API Client
↓
Backend

rather than:

Page
↓
Everything

Keep pages thin and feature modules responsible for their own domain UI.